

```

/**
 * Core Database Access Library
 *
 * Provides a standardized way to log DML, errors, and success states with
 * ANSI color support and environmental filtering.
 *
 * @module QueryUtils
 * @file db/query.util.ts
 *
 * Acknowledgments:
 * Architecture for session-reactive logging and atomic DML utility developed in collaboration
 * with AI on Google Search, powered by the Gemini family of models.
 *
 * @copyright 2018-2026, Dennis Jorgenson
 */
import type { IUser } from "#db/interfaces/user";

import { Select } from "#db/query.util";
import { setSession } from "#app/session";

// src/cli/pages/login.ts
import { isEqual } from "#lib/std.util";
import { hashPassword } from "#db/interfaces/user";

import * as p from "@clack/prompts";

const module = `Ligon`;

export const Logon = async (context = ""): Promise<boolean> => {
  context = `${context}.Logon`;
  p.intro(`\x1b[36m 2026 Engine: Security Ignition \x1b[0m`);

  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

  const group = await p.group(
    {
      // 1. Collect Identity
      username: () => {
        p.text({
          message: "Identity (Username or Email):",
          placeholder: "user@engine.com",
          validate: (v) => (!v ? "Required" : undefined),
        }),
      },

      // 2. Conditional Email Collection
      email: ({ results }) => {
        // If identity is already an email, skip this prompt
        if (emailRegex.test(results.username as string)) {
          return;
        }
        return p.text({
          message: "Primary Email Address:",
          validate: (v) => (!emailRegex.test(v!) ? "Invalid email" : undefined),
        });
      },

      password: () => p.password({ message: "Set Secret:" }),
    },
    {
      onCancel: () => {
        p.cancel("Aborted.");
        process.exit(0);
      },
    },
  );

  /**
   * 3. EXTRACTION LOGIC
   * We derive the actual username and email from the results
   */
  const isEmail = emailRegex.test(group.username);
  const finalEmail = isEmail ? group.username : group.email;
  const finalUsername = isEmail
    ? group.username.split("@")[0] // Extract 'john' from 'john@work.com'
    : group.username;

  // Now pass these to your DML Insert
  console.log(` Provisioning: ${finalUsername} / ${finalEmail}`);

  const s = p.spinner();
  s.start("Verifying Credentials...");

  // 1. Fetch User
  // CASTING: group.username is 'string | symbol', we cast to string for the query
  const result = await Select<IUser>({ username: finalUsername, email: finalEmail as string }, { table: "vw_users" }, context);
  const { data, meta, ...detail } = result;
  console.log(` [Info] ${context}`, { detail, data });
  const user = result.data?.[0];

  if (!user) {
    s.stop("Identity Rejected");
    return false;
  }

  // 2. Structural & Null Checks
  if (!user || !user.hash || !user.password) {
    s.stop("Identity Rejected");
    return false;
  }

  // 3. Multi-Hash Re-calculation
  const inputKey = hashPassword({
    username: user.username,
    email: user.email,
    password: group.password as string,
    hash: user.hash, // ERD shows 'hash' column as binary/varbinary
  });

  // 4. Comparison Logic
  // Using Buffer.from handles the raw binary data shown in your ERD
  if (inputKey && isEqual(Buffer.from(user.password), inputKey)) {
    /**
     * ERD CHECK: 'state' table holds the 'status'
     * Ensure your view joins user.state -> state.state and aliases state.status as auth_status
     */
    if (user.status !== "Enabled") {
      s.stop(` Access Denied: ${user.status}`);
      return false;
    }
  }
}

```

```

s.stop(`Authenticated: ${user.username}`);

setSession({
  user: user.user, // UUID/PK from ERD
  username: user.username,
  isAdmin: (user.auth_rank ?? 0) > 90, // auth_rank is an int in your user table
  alias: "BOOT",
});

return true;
}

s.stop("Secret Mismatch");
return false;
};

/*
export const Logon = async (log: ILogger): Promise<boolean> => {
  p.intro(`\x1b[36m 2026 Engine: Security Ignition \x1b[0m`);

  const group = await p.group(
    {
      username: () => p.text({
        message: "Identity (User/Email):",
        validate: (value) => !value ? "Identity required" : undefined
      }),
      password: () => p.password({
        message: "Secret:",
        validate: (value) => !value ? "Password required" : undefined
      }),
    },
    {
      onCancel: () => {
        p.cancel("Ignition Aborted.");
        process.exit(0);
      },
    },
  );

  const s = p.spinner();
  s.start("Verifying Credentials...");

  // 1. Fetch User
  // CASTING: group.username is 'string | symbol', we cast to string for the query
  const result = await Select("vw_user_auth", { username: group.username as string }, log);
  const user = result.data?.[0];

  if (!user) {
    s.stop("Identity Rejected");
    return false;
  }

  /**
   * 2. THE MULTI-HASH RE-CALCULATION
   * FIX: Ensure we handle the conversion of DB binary types to Uint8Array
   * for the hashPassword utility.
   */
  // const inputKey = hashPassword({
  //   username: user.username,
  //   email: user.email,
  //   password: group.password as string,
  //   // Ensure the salt from DB is passed as a Uint8Array
  //   hash: user.hash ? new Uint8Array(user.hash) : undefined,
  // });

  // // 3. Compare Input Key against DB Password
  // // FIX: Using Buffer.from ensures we are comparing like-for-like memory chunks
  // if (inputKey && isEqual(Buffer.from(user.password), inputKey)) {

  // // 4. Status Check
  // if (user.auth_status !== "Authorized") {
  //   s.stop(`Access Denied: Account is ${user.auth_status}`);
  //   return false;
  // }

  // s.stop(`Authenticated: ${user.username}`);

  // setSession({
  //   user: user.user,
  //   username: user.username,
  //   isAdmin: Number(user.auth_rank) > 90,
  //   alias: "BOOT",
  // });

  // return true;
  // }

  // s.stop("Secret Mismatch");
  // return false;
  // };

```